Restaurant Ordering Technical Design

Repository URL: Publication pending. Insert the real public GitHub URL here before submission.

Live application: Local demonstration at http://localhost:5000. No public deployment is claimed.

Author: Anuj Kumar Singh

Scope and architectural decision

Dum & Co. implements the restaurant brief's complete core: customer menu and cart, simulated payment, central menu administration, an outlet kitchen display and AI preparation estimates. Additional depth includes coupons, loyalty, party enquiries, outlet stock and hours, history-based recommendations and an operations overview. The demonstration uses fictional data and external-system mocks as permitted by the brief.

The application uses React with Vite and Tailwind, Express on Node.js, MongoDB with Mongoose, and the ws WebSocket library. Its layout follows the author's hostel-seat-allocation project: backend config, controllers, middleware, models, routes, services and algorithms; frontend pages, components, services and utils. CommonJS on the backend and ES modules on the frontend preserve familiar boundaries.

A single backend process is a deliberate choice for this time-boxed slice. HTTP owns durable reads and writes, while WebSocket announces scoped changes and clients refetch authorized records. MongoDB transactions protect payment and reward state. There is no Kafka or Redis dependency for this local demonstration; distributed notification delivery is a documented scaling step.

Development starts a persistent loopback-only MongoDB replica set automatically. Production requires a real transaction-capable replica set and a configured secret. Express can serve the compiled frontend and APIs on one origin, avoiding cross-site session complexity.

Roles and boundaries

Customers can register, browse, order, pay in demo mode, read their own history and request party orders. Kitchen staff are assigned one outlet and can work only with that outlet's orders and stock. Central managers edit the shared menu, view all outlets, override preparation promises and review catering enquiries. The manager role is an explicit assumption needed for central menu administration; the core brief names customer and kitchen staff roles.

Public registration strips unrecognized fields and always assigns customer. Password hashes are never returned. Staff access is checked on the server for every protected route, not merely hidden in navigation. Order queries include the authenticated customer ID or assigned outlet ID.

Data model

Entity	Principal stored information
User	Name, email, password hash, role, assigned outlet, reward balance, consent and optional marketing opt-in
Outlet	City, address, illustrative licence ID, coordinates, delivery radius, minimum prep time and station capacity
MenuItem	Description, dietary and allergen data, photos, base and city prices, portions, add-ons, hours and paused outlets
Order	Customer and outlet references, immutable item/price snapshot, totals, payment and fulfilment state, model estimate, promises and event history
PartyRequest	Customer, outlet, guest count, requested time, notes and staff decision

User email, public order number and the pair of customer ID with checkout idempotency key have unique indexes. Kitchen reads are indexed by outlet, payment state, stage and creation time. The item snapshot preserves what was ordered even if the central menu changes later. The server always computes money in integer paise.

Main order flow

The customer chooses an outlet and constructs a bag of item IDs, portions, add-ons and quantities. The server validates the payload, reloads menu prices and checks outlet availability and hours. Delivery requires coordinates inside the outlet radius and an address; pickup needs neither. Coupon savings and eligible reward redemption are calculated server-side, with GST already included at the brief's specified rate.

The server saves an awaiting-payment order with a unique retry key and a request hash. Reusing the key with the same validated payload returns that order. Reusing it for a different cart returns a conflict. The payment simulator supports success, failure and retry without collecting real payment credentials or charging money.

On success, a transaction rechecks stock, conditionally deducts available reward points, records the payment and confirms the order. An already-paid guard makes repeated callbacks harmless. A confirmed ticket reaches only the matching kitchen. Allowed stages are confirmed, preparing, ready and handed off. Invalid stage jumps fail. Handoff credits reward points once; cancellation before readiness returns redeemed points atomically.

Live updates and recovery

WebSocket upgrades validate the session cookie and Origin. Change events contain an order ID rather than a full customer record. Recipients are restricted to its customer, assigned outlet kitchen and central manager. A heartbeat closes dead or expired connections. The browser reconnects after three seconds, refetches on connection, and polls order data every fifteen seconds as a backup. Persisted database state remains the source of truth if a notification is missed.

Preparation model and recommendations

The AI component is a small ridge-regression model for numerical preparation prediction. Its features are an intercept, longest item preparation time, queued portions per cooking station, order portion count and dinner-peak indicator. The model learns coefficients by solving regularized normal equations with pivoted elimination. The code and training generator are inspectable in backend/src/algorithms/prepTime.js.

The deterministic synthetic fixture contains 480 examples: 384 train the model and 96 remain held out. The current held-out mean absolute error is approximately 1.74 minutes. This result applies only to synthetic fixtures generated from explicit assumptions; it is not evidence of real restaurant accuracy. Before production, replace fixtures with real completed-order observations, separate validation by time and outlet, and examine prediction drift.

For typical inputs, the customer promise is the larger of the outlet minimum and the model estimate plus a two-MAE buffer. Queue load above 20 portions per station, more than 12 order portions, or item prep outside 8 to 33 minutes triggers a conservative rules fallback and a low-confidence flag. Estimates are never presented as guaranteed model accuracy. Staff can override with a reason, but cannot undercut the outlet minimum.

An order more than ten minutes beyond its promise creates a persisted late event and an alert on the kitchen/manager view. The customer sees the event and can receive a revised promise from staff. A periodic worker checks overdue orders every thirty seconds and marks each escalation once. External SMS and WhatsApp delivery are simulated by the persisted order-event record and in-app updates; no external messages are sent.

Optional recommendations learn a content-based taste profile from the customer's paid, non-cancelled orders. Category, diet and spice features form a weighted profile, then cosine similarity and repeat-item counts rank available dishes. New customers see no personalized section without history. Suggestions never modify the bag automatically and are filtered by outlet stock and hours. This is a separate content recommender, not a language-model claim.

Security and privacy

Sessions use signed HS256 JWTs with issuer and expiry verification in HttpOnly, SameSite=Lax cookies. Production cookies require HTTPS. Bcrypt hashes passwords; Zod validates input shapes, limits and ranges. Exact-origin CORS and write-origin checks constrain browser requests. JSON bodies have a size limit. Login and API rate limits, Helmet headers and a content security policy provide basic hardening.

No real personal profile, payment secrets or database credentials are placed in source. Local generated secrets and database files are ignored. Registration records service consent, while marketing opt-in is separately optional. Demo account passwords are intentionally public for fictional data only. Production requires a reviewed retention policy, account management and real notification/payment providers.

UI decisions and key screens

Screen	Design decision and purpose
Menu and item details	Large illustrative food photos, restrained green and cream palette, serif headings and clear prices support appetite and scanning. Filters and portion choices keep the menu short.
Bag and demo checkout	Clear cost breakdown, inclusive tax, visible discounts and explicit simulation avoid payment ambiguity. Failed payment has a retry path.
Customer tracking	A prominent promised time and plain preparation timeline reduce uncertainty. Event history explains changes or delays.
Kitchen board	Separate stage columns, large ticket content, timers and warm late-order colours make prioritization visible. Tablet breakpoints reduce column density.
Central menu editor	Structured fields for prices, hours, allergens, portions and extras keep validation explicit. Native dialogs contain keyboard focus.
Operations and party requests	Compact metrics, recent outlet records and a staff decision queue give optional depth without replacing billing or inventory systems.

The header keeps outlet selection available on phones. Pages include loading, empty, failure and disconnected states. Menu images are bundled and lazily loaded, removing runtime dependency on an image provider. Native form controls and dialogs, labelled inputs, keyboard focus styling and reduced-motion support improve accessibility. FSSAI IDs are visible demo placeholders and must be replaced before launch. Food images are illustrative rather than photographs of the fictional client's dishes.

Verification and practical limits

The backend test suite has 21 passing tests against an isolated MongoDB replica set. Coverage includes paise arithmetic, inclusive tax, coupons, menu hours and stock, consent, role escalation, object-level access, invalid input, CSRF-origin rejection, checkout retries, declined payment, concurrent successful payment retries, single reward credit, invalid stage transitions, model bounds, override minimums, duplicate late alerts, authenticated WebSocket events, recommendations and menu edit validation.

The production frontend builds successfully. Browser verification exercised portions and add-ons, customer sign-in, coupon checkout, failed-then-successful payment, live tracking, matching kitchen arrival, preparation stage change, staff promise override and ready-for-pickup. Desktop and phone layouts are inspected separately. Dependency audits reported no known production dependency vulnerabilities at the time of this build.

This evidence establishes a working local demonstration, not sustained production throughput. The stated peak of 1,200 orders per hour across outlets has not been load-tested. In-process broadcasts and rate limits assume one API process. Database writes survive restarts, but a crash can lose a live notification; reconnection refetch and polling recover current state. Production should use an outbox and shared pub/sub for durable multi-instance delivery.

Assumptions and scope decisions

A1. Dum & Co. is a fictional brand. Three sample outlets represent the client's broader chain of 14 outlets and three cloud kitchens. Seeded orders preserve the three sample order IDs, outlet names, item totals and preparation states from the brief; their clocks are relative to the first seed so the demonstration is usable.

A2. Menu prices, extra items, portion multipliers, outlet coordinates and radii, cooking capacities and minimum prep times are illustrative assumptions. Every image is illustrative stock photography, not evidence of a real dish. Licence identifiers explicitly say DEMO. Client production details must replace these values.

A3. The training generator, dinner window of 19:00 to 22:59 IST, synthetic feature ranges and uncertainty thresholds are prototype assumptions. A two-MAE buffer is a heuristic, not a calibrated probability interval. They require real validation and operations approval before launch.

A4. Coupon DUM10 gives ten percent off up to INR 100. One reward point equals INR 1; an order earns one point per INR 100 paid after handoff, and redemption is capped at 100 points per order. Points are credited or returned atomically. These are explicit business-rule assumptions, not client-specified policy.

A5. Party requests require at least ten guests and forty-eight hours' notice. Central managers or assigned kitchen staff act as the catering approver for this demonstration. No quote or confirmation message is sent externally.

A6. Demo delivery is radius-checked, has no delivery fee and ends at simulated rider handoff. Real route planning, rider apps, inventory replacement and delivery-marketplace integration remain out of scope, consistent with the brief. Live SMS, WhatsApp and payment integrations are mocked.

Next work and release steps

Production priorities are real kitchen observations and calibrated estimates; real outlet/menu/licence information; real payment verification and refunds; a durable notification outbox; Redis fan-out and distributed limits for multiple API instances; data retention and deletion workflows; operational monitoring, backup restoration and capacity tests. Client integrations should remain behind clearly defined adapters rather than mixed into pricing logic.

For reviewer delivery, publish the repository when authorized, preserve the actual local commit history and replace the repository line at the top of this document with its real public URL. Optionally deploy the single-origin app and add its URL. The submission ZIP should contain only the final technical-design PDF and deployment.md, under 25 MB. No public repository, deployment or assessment submission is claimed by this local build.

Reference and run commands

The functional source is the expanded ISOCODE restaurant brief at workspace.isocodelabs.com. The folder reference is github.com/Anuj8506/hostel-seat-allocation. Official implementation references include mongoosejs.com/docs/connections.html, github.com/websockets/ws and vite.dev/guide. Full asset provenance is in docs/ASSETS.md.

Run npm run setup, then npm run dev for development. Run npm test for the isolated integration suite, npm run build for the frontend, and npm start for the single-server demonstration. Reviewer demo accounts and environment-variable names are in deployment.md. The training command is npm run train --prefix backend.